FileCopy

User Guide

v2026.05.12.0005

CRC32-verified, crash-safe parallel file copy with scheduling & conflict resolution

Windows 10/11 • .NET 10 • WPF/MVVM

1. Overview

FileCopy is a professional file-copy utility with built-in integrity verification (CRC32), crash-safe resume, parallel transfer lanes, job scheduling, and six conflict-resolution modes. It targets power users and content creators who need reliable, auditable file operations with minimal manual intervention.

All file operations produce a sidecar .crc file alongside each destination file, enabling post-copy verification without re-reading the source.

2. System Requirements

Component	Requirement
OS	Windows 10 v1903 or Windows 11
Runtime	.NET 10 Desktop Runtime (x64)
RAM	128 MB minimum
Storage	10 MB for application; sidecar files add ~1 KB per copied file
Permissions	Read access to source; Write access to destination

3. Installation

Run the FileCopy installer or extract the portable ZIP to any location. Settings are stored in %APPDATA%\FileCopy\settings.json. Job lists (.fclist files) are stored wherever you save them.

No elevation required for standard copy operations. Elevation is requested automatically if the destination path requires it.

4. Interface Tour

Zone	Location	Purpose
1 Identity/Scale	Top bar	App name, version, global DPI scale (100/125/150/200%)
2 NavRow	Row 2	Jobs | Transfer | Scheduler | Settings | Presets | Log
3 Toggle/Preview	Row 3	Start/Pause/Stop transfer controls + overall progress bar
4 Main / Sliders	Centre	Active panel: job list, conflict options, CRC settings, parallel lanes slider
5 Presets 1–5 / Save	Bottom bar	Quick-access job presets + Save; resize grip bottom-right
6 Status Bar	Very bottom	Current file name, speed (MB/s), ETA, error count

5. Quick Start

1. Click Jobs → Add Job (or drag folders onto the job list).

2. Set Source and Destination paths. Choose a Conflict Mode (default: Ask).

3. Optionally set parallel lanes (1–8) and enable CRC verification.

4. Click Start Transfer (Zone 3) or press Ctrl+Enter.

5. Monitor progress in the status bar. On completion, a summary report shows file count, total size, CRC pass/fail, and any skipped files.

6. Conflict Resolution

When a file already exists at the destination, FileCopy applies the selected conflict mode:

Mode	Behaviour
Ask	Pause and prompt the user for each conflict (default)
Skip	Leave existing file untouched; log the skip
Overwrite	Replace existing file unconditionally
Overwrite if Newer	Replace only if source modification date is later
Rename	Append (1), (2), … to the destination filename
Merge	For folders: descend into both; for files: fall back to Ask mode

■ Conflict mode is saved per job and can be overridden globally in Settings → Default Conflict Mode.

7. CRC32 Integrity Verification

When CRC Verification is enabled (Settings or per-job toggle), FileCopy:

1. Computes CRC32 of each source file using System.IO.Hashing.Crc32 during the read pass.

2. Writes a sidecar file (filename.ext.crc) in the destination folder containing the hex CRC value.

3. After writing, reads the destination file and recomputes CRC32, comparing against the source value.

4. Any mismatch is flagged as a Verification Error — the file is quarantined to a _failed subfolder and logged.

```
Sidecar format: \t\t\t
```

■ CRC32 via System.IO.Hashing is hardware-accelerated on CPUs with SSE4.2/ARM CRC extensions.

8. Crash-Safe Resume

FileCopy writes a .fcresume checkpoint file to the destination folder at configurable intervals (default: every 64 MB or every 30 seconds). If the application crashes or the system powers off mid-copy:

1. On next launch, FileCopy detects orphaned .fcresume files and offers to resume.

2. Resume re-reads the checkpoint to determine the last verified byte offset per file.

3. Copy resumes from that offset; already-copied bytes are not re-transferred.

4. CRC is recomputed over the entire completed file after resume to ensure integrity.

■ Resume is per-file. Completed files in a multi-file job are not re-copied on resume.

9. Parallel Transfers

The Parallel Lanes slider (1–8, Zone 4) controls how many files copy simultaneously. Each lane is an independent async stream with its own progress tracking.

Lanes	Best Use Case
1	Single large file, HDD source (prevents seek thrashing)
2–4	Mixed file sizes, SSD source/destination
5–8	Many small files, NVMe or network share source

■ Network copies default to 2 lanes. Increase only if the destination supports concurrent writes (NAS with RAID, cloud storage).

10. Scheduler

Open Scheduler (NavRow) to create time-triggered copy jobs. Each schedule entry has a start time, recurrence, source job, and optional stop-after duration.

Field	Values	Example
Run At	HH:MM 24h	02:30
Recurrence	Once / Daily / Weekly / Custom cron	Weekly
Days	Mon–Sun checkboxes	Mon, Wed, Fri
Stop After	Minutes (0 = run to completion)	120
Conflict Mode	Override per schedule	Overwrite if Newer

■ Schedules run even if the FileCopy window is minimised to the system tray.

11. Presets

Five preset slots (Zone 5) each store a complete job configuration: source, destination, conflict mode, parallel lanes, CRC setting, and schedule. Load a preset with a single click or via Ctrl+Shift+1–5.

12. Job Files (.fclist)

Job lists can be exported to .fclist files (JSON format) for sharing and scripting. Use File → Export Job List or run FileCopy.exe --run jobfile.fclist for unattended operation.

```
FileCopy.exe --run backup.fclist --conflict overwrite --lanes 4 --verify
```

13. Keyboard Shortcuts

Shortcut	Action
Ctrl+Enter	Start / Resume transfer

Ctrl+P	Pause transfer
Ctrl+Q	Stop and cancel transfer
Ctrl+J	Open Jobs panel
Ctrl+L	Open Log panel
Ctrl+Shift+1–5	Load preset 1–5
Ctrl+Z / Ctrl+Y	Undo / Redo job configuration changes
F5	Refresh file size estimates
Alt+S	Open Settings
Alt+T	Cycle theme
Alt+H	Hide to tray

14. Settings Reference

Setting	Type	Default	Description
UiScale	Enum	1.0	1.0/1.25/1.5/2.0
DefaultConflictMode	Enum	Ask	Global conflict mode override
ParallelLanes	Int	2	Default lane count (1–8)
EnableCrc	Bool	true	Enable CRC32 verification by default
ResumeCheckpointMb	Int	64	MB between resume checkpoints
ResumeCheckpointSec	Int	30	Seconds between resume checkpoints
SidecarExtension	String	.crc	Sidecar file extension
SidecarLocation	Enum	Alongside	Alongside/CentralFolder
ShowSpeedMBs	Bool	true	Show MB/s in status bar
ShowEta	Bool	true	Show ETA in status bar
MinimiseToTray	Bool	false	Minimise to system tray
ThemeName	String	Dark	One of 20 built-in themes
ThemeAuto	Bool	false	Follow Windows light/dark
LogLevel	Enum	Warning	Debug/Info/Warning/Error
LogRetentionDays	Int	30	Days to keep log files

15. Themes

20 built-in themes: Dark, Light, Nord, Dracula, Catppuccin Mocha, Catppuccin Latte, Ayu Dark, Ayu Light, Ayu Mirage, Cobalt2, GitHub Light, Gruvbox Dark, Monokai, One Dark, Rose Pine, Solarized Dark, Solarized Light, Tokyo Night, Warm Light, High Contrast. Switch via Identity bar or Alt+T.

16. Accessibility

Criterion	Level	Status
1.4.6 Contrast (AAA)	AAA	All text ≥7:1 on all 20 themes
2.5.5 Target Size	AA	All controls ≥44×44 px
2.4.7 Focus Visible	AA	2 px Gold/Cyan ring on keyboard focus
1.4.1 Use of Colour	A	Error states use icon + label, not colour alone
2.1.1 Keyboard	A	Full keyboard navigation; no mouse-only actions

17. Troubleshooting

Symptom	Fix
CRC mismatch errors	Check source media for read errors; try 1 lane to reduce I/O contention
Transfer stalls at 0 B/s	Source or destination path disconnected; check network/drive
Resume not offered	Checkpoint interval too long — lower ResumeCheckpointMb in Settings
High memory usage	Reduce parallel lanes; each lane buffers up to 16 MB
Schedule doesn't trigger	Check system clock; verify FileCopy is running (tray icon)
Cannot write to destination	Path requires elevation — relaunch FileCopy as Administrator

18. Log Files

Logs: %APPDATA%\FileCopy\logs\filecopy_YYYY-MM-DD.log

Each completed job also generates a summary report: %APPDATA%\FileCopy\reports\job_.json

```
Log path: C:\Users\\AppData\Roaming\FileCopy\logs\
```

19. Build Notes

Version v2026.05.12.0005 built with .NET 10 SDK, C# 14, WPF/MVVM. CRC32 via System.IO.Hashing (hardware-accelerated). JSON source generation for settings serialisation. Versioning: yyyy.MM.dd.NNN.

© 2026 Matthew Pittman. All rights reserved.